

**TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHẦN HIỆU TẠI TP.HCM
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



JUnit nâng cao — Mockito

Buổi 6 / 12 — Môn: Kiểm thử Phần mềm

Ngày 1 tháng 8 năm 2026

Nội dung buổi học

- 1 Vấn đề: unit test một class có dependency
- 2 Test double: dummy, stub, fake, spy, mock — và Mockito
- 3 Thiết lập: `@ExtendWith`, `@Mock`, `@InjectMocks`
- 4 Stubbing: `when/thenReturn`, `thenThrow`, argument matcher
- 5 Phân tích ca kiểm thử: `InterceptionServiceImpl.evaluate()`
- 6 Verify: `times`, `never`, `verifyNoInteractions`, `InOrder`
- 7 ArgumentCaptor: bắt và kiểm tra đối tượng được lưu
- 8 Kiểm thử exception: `assertThrows`, `assertThatThrownBy`
- 9 Kiểm thử phát sự kiện: `RiskRecomputedEvent`
- 10 Tổ chức test: `@Nested`, `@DisplayName`, helper builder
- 11 `@Mock` vs `@MockitoBean` (Spring)
- 12 Anti-patterns — Thực hành — Bài nộp

Vị trí trong đề cương

Chương 4 — Phân tích một ca kiểm thử với JUnit

Buổi 6 là buổi thứ hai của Chương 4: từ JUnit 5 cơ bản (Buổi 5) sang kiểm thử class có **dependency** bằng **Mockito**.

Buổi	Chương	Chủ đề
5	C4 JUnit	JUnit 5 cơ bản: lifecycle, assertions, AssertJ
6	C4 JUnit	Mockito: mock, stub, verify, captor
7	C4 (mở rộng)	Integration test: Spring Boot, Testcontainers

Mục tiêu buổi học

Đọc hiểu và tự viết được bộ unit test Mockito hoàn chỉnh cho một service thật của DigiShield — đúng như bộ test đang chạy trong repo.

Ôn tập Buổi 5 — và giới hạn của nó

Buổi 5 đã test được

- `StubAiClient.classify()` — hàm thuần, không dependency
- `ReportingServiceImpl.ageLabel()` — tính nhãn thời gian
- `Assertions`, `AssertJ`, `@ParameterizedTest`

Câu hỏi mở đầu

Làm sao unit test

`InterceptionServiceImpl`

`.evaluate()` khi nó cần **đọc watchlist từ database** và **ghi lại sự kiện can thiệp?**

```
public InterceptionServiceImpl(
    AccountWatchEntryRepository watchRepository,
    InterventionEventRepository eventRepository,
    Messages messages) { ... } // 3 dependency!
```

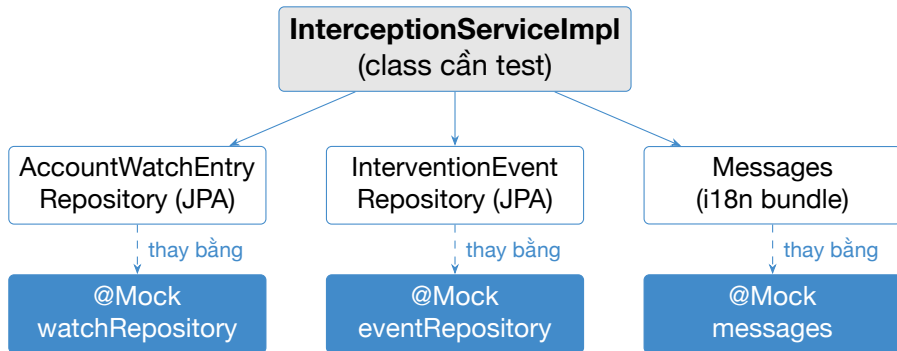
Vì sao không dùng database thật trong unit test?

- **Chậm:** khởi động PostgreSQL/H2 + Spring context mất vài giây; unit test cần chạy trong **mili-giây** (hàng trăm test mỗi lần build).
- **Không ổn định:** dữ liệu tồn dư giữa các test, thứ tự chạy ảnh hưởng kết quả (vi phạm tính *isolated* của FIRST).
- **Khó dựng tình huống:** làm sao ép repository ném exception, hay trả về đúng một bản ghi watchlist "CONFIRMED"?
- **Sai mức kiểm thử:** unit test kiểm tra **logic quyết định** của service; việc JPA đọc/ghi đúng là chuyện của integration test (Buổi 7).

Giải pháp

Thay dependency thật bằng **test double** — đối tượng "đóng thế" do ta điều khiển hoàn toàn. Mockito là thư viện tạo test double phổ biến nhất trong hệ sinh thái Java.

Dependency của InterceptionServiceImpl



Cả 3 dependency đều được mock

Không cần PostgreSQL, không cần Spring context, không cần file message bundle. Test chạy hoàn toàn **in-memory**.

Phân loại test double (G. Meszaros [7])

Loại	Mô tả	Ví dụ DigiShield
Dummy	Chỉ để lấp tham số, không bao giờ được dùng thật	UUID truyền cho DTO
Stub	Trả về câu trả lời định sẵn cho lời gọi	repository trả <code>Optional.of(hit)</code>
Fake	Cài đặt thật nhưng đơn giản hóa	StubApiClient thay Claude API
Spy	Bọc đối tượng thật, ghi lại lời gọi	@Spy trên service thật
Mock	Được lập trình cả kỳ vọng về hành vi : gọi gì, mấy lần	<code>verify(eventRepository).save(...)</code>

Lưu ý thuật ngữ

Trong Mockito, đối tượng @Mock dùng được như **cả stub lẫn mock**: `when(...)` để stub, `verify(...)` để kiểm tra hành vi.

Stub vs Mock — hai kiểu xác minh

State verification (dùng stub)

- Cho dependency trả dữ liệu định sẵn
- Assert trên **kết quả trả về**
- Ví dụ: watchlist hit → decision phải là PAUSE

Behaviour verification (dùng mock)

- Kiểm tra service **đã gọi** dependency đúng cách
- Assert trên **tương tác**
- Ví dụ: `eventRepository.save()` được gọi đúng 1 lần

Nguyên tắc chọn

Ưu tiên state verification khi method **trả về giá trị**; dùng behaviour verification cho **side effect** (ghi DB, publish event, gửi email) — vì không có giá trị trả về nào để assert.

Mockito là gì?

Mockito [5]

Thư viện mã nguồn mở (site.mockito.org) tạo test double cho Java. Sinh **proxy class** lúc runtime (ByteBuddy) thay cho interface / class thật; mặc định mọi method trả null / 0 / false / collection rỗng.

- Đi kèm sẵn trong `spring-boot-starter-test` — DigiShield không cần khai báo thêm dependency nào.
- Tích hợp JUnit 5 qua `MockitoExtension` (gói `mockito-junit-jupiter`).
- Mock được interface **và** class thường; class/method `final`, `static` cần `mockito-inline` (hạn chế dùng).

Trong repo DigiShield

176 unit test hiện có đều theo cùng một khuôn: `MockitoExtension` + `@Mock`/`@InjectMocks`/`@Captor` + `AssertJ`.

Ba annotation cốt lõi

Annotation	Ý nghĩa
@Mock	Tạo mock cho field — thay dependency thật
@InjectMocks	Tạo instance thật của class cần test, tự tiêm các @Mock vào qua constructor
@Captor	Tạo ArgumentCaptor để bắt argument (xem sau)

```
@ExtendWith(MockitoExtension.class) // kích hoạt Mockito
class InterceptionServiceImplTest {
    // @Mock -> the vào cho 3 dependency
    // @InjectMocks -> new InterceptionServiceImpl(
    //                     watchRepository, eventRepository,
    //                     messages)
}
```

Quên @ExtendWith(MockitoExtension.class)?

Các field @Mock sẽ là null → NullPointerException ngay dòng đầu tiên của test.

Khai báo thật trong InterceptionServiceImplTest (1/2)

Trích nguyên văn từ modules/interception/src/test/...:

```
@ExtendWith(MockitoExtension.class)
class InterceptionServiceImplTest {

    private static final UUID TENANT_ID = UUID.fromString(
        "11111111-1111-1111-1111-111111111111");
    private static final String DEST_ACCOUNT = "1234567890";

    @Mock
    private AccountWatchEntryRepository watchRepository;

    @Mock
    private InterventionEventRepository eventRepository;
```

Khai báo thật trong InterceptionServiceImplTest (2/2)

```
@Mock
private Messages messages;

@InjectMocks
private InterceptionServiceImpl interceptionService;

@Captor
private ArgumentCaptor<InterventionEvent> eventCaptor;
```

Setup và teardown — trích test thật

```
@BeforeEach
void setUp() {
    // Service doc tenant tu TenantContext (ThreadLocal)
    TenantContext.set(TENANT_ID.toString());
    // save() tra ve chinh doi tuong duoc luu
    lenient().when(eventRepository.save(
        any(InterventionEvent.class)))
        .thenReturn(inv -> inv.getArgument(0));
    // messages.get(key) tra ve chinh key
    lenient().when(messages.get(anyString()))
        .thenReturn(inv -> inv.getArgument(0));
}

@AfterEach
void tearDown() {
    TenantContext.clear(); // don ThreadLocal sau moi test
}
```

Vì sao cần `lenient()`?

Strict stubbing (mặc định từ Mockito 2+)

Mockito **fail test** nếu một stub được khai báo mà không được dùng: `UnnecessaryStubbingException`. Mục đích: phát hiện test "chết" và stub thừa.

- Stub trong `@BeforeEach` áp dụng cho **mọi** test của class.
- Nhưng test `checkAccount_...` không hề gọi `eventRepository.save()` → stub đó "thừa" với test này.
- `lenient()` đánh dấu: stub này *được phép* không dùng.

Kinh nghiệm

Chỉ `lenient()` cho stub dùng chung trong `@BeforeEach`; stub riêng của từng test vẫn để strict — nó là "máy dò" stub rác miễn phí.

Stubbing cơ bản: when / thenReturn (1/2)

```
// Hai kick ban THAY THE nhau -- moi kick ban dat trong
// MOT test rieng (dat chung: stub sau ghi de stub truoc,
// strict stubbing bao UnnecessaryStubbing)

// Kick ban A: "khi duoc hoi tai khoan nay, tra ve mot
// ban ghi watchlist muc CONFIRMED"
AccountWatchEntry watchHit = new AccountWatchEntry(
    UUID.randomUUID(), TENANT_ID, WatchType.BANK_ACCOUNT,
    DEST_ACCOUNT, RiskLevel.CONFIRMED, "fraud-feed");

when(watchRepository
    .findByTenantIdAndValue(TENANT_ID, DEST_ACCOUNT))
    .thenReturn(Optional.of(watchHit));
```

Stubbing cơ bản: when / thenReturn (2/2)

```
// Kịch bản B: không có trong watchlist
when(watchRepository
    .findByTenantIdAndValue(TENANT_ID, DEST_ACCOUNT))
    .thenReturn(Optional.empty());
```

Đọc như một câu điều kiện

`when(mock.method(args)).thenReturn(value)` — "khi method được gọi với args này thì trả về value". Chưa stub → trả mặc định: object là null, nhưng `Optional` là `Optional.empty()`, collection là rỗng.

thenThrow, thenAnswer, stub nhiều lần gọi

```
// Gia lap DB loi: lan gọi nao cung nem exception
when(watchRepository.findByTenantIdAndValue(any(), any()))
    .thenThrow(new DataAccessResourceFailureException(
        "connection lost"));

// Lan 1 loi, lan 2 tra ve rong (mo phong retry)
when(watchRepository.findByTenantIdAndValue(any(), any()))
    .thenThrow(new RuntimeException("timeout"))
    .thenReturn(Optional.empty());

// thenAnswer: tinh ket qua tu chinh argument --
// dung trong test that của DigiShield cho save():
when(eventRepository.save(any(InterventionEvent.class)))
    .thenAnswer(inv -> inv.getArgument(0));
```

thenAnswer cho repository

JPA save () trả về entity đã lưu. Stub "trả về chính argument" mô phỏng đúng hợp đồng đó mà không cần database.

doReturn vs thenReturn, doThrow cho void

```
// deleteById() tra ve void -> KHONG viet duoc
// when(repo.deleteById(id)).thenThrow(...) // sai cu phap!

// Dang do...when danh cho void method:
doThrow(new IllegalStateException("locked"))
    .when(watchRepository).deleteById(any(UUID.class));

// doReturn: không gọi method that -> bắt buộc với @Spy
doReturn(Optional.empty())
    .when(spyService).checkAccount(anyString());
```

<code>when(...).thenReturn(...)</code>	<code>doReturn(...).when(...)</code>
--	--------------------------------------

Cách viết chuẩn, type-safe

Dùng khi method **void** (doThrow, doNothing) hoặc khi stub trên @Spy

Gọi method trên mock (vô hại)

Không thực thi method thật

Argument matcher: any(), eq(), anyString()

```
// Gia tri cu the: chỉ khớp đúng tham số này
when(watchRepository
    .findByTenantIdAndValue(TENANT_ID, DEST_ACCOUNT))
    .thenReturn(Optional.of(watchHit));

// Matcher: khớp mọi giá trị
when(messages.get(anyString()))
    .thenReturn(inv -> inv.getArgument(0));

// Tron matcher và giá trị raw -> PHẢI bọc eq():
when(riskSignalRepository
    .findByTenantIdAndUserIdAndOccurredAtAfter(
        eq(TENANT_ID), eq(userId), any(Instant.class)))
    .thenReturn(List.of());
```

Quy tắc bắt buộc

Trong **một** lời gọi stub/verify: hoặc **tất cả** tham số là giá trị raw, hoặc **tất cả** là matcher. Trộn lẫn → `InvalidUseOfMatchersException` lúc chạy.

Mã nguồn evaluate() — gom tín hiệu (1/2)

modules/interception/.../InterceptionServiceImpl.java:

```
@Override
public InterventionDecision evaluate(EvaluateRequest request) {
    UUID tenantId = TenantContext.requireUuid();

    List<String> signals = new ArrayList<>();
    if (request.onCall()) {           // đang nghe điện thoại?
        signals.add("ON_CALL");
    }
    if (request.newPayee()) {         // người nhận mới?
        signals.add("NEW_PAYEE");
    }

    Optional<AccountWatchEntry> hit = watchRepository
        .findByTenantIdAndValue(tenantId,
                                request.destAccount());
    boolean watchlistHit = hit.isPresent();
    if (watchlistHit) {
        signals.add("WATCHLIST_HIT");
    }
}
```

Mã nguồn evaluate() — ra quyết định (2/2)

```
Decision decision;
String message;
if (request.onCall() && request.newPayee()
    && watchlistHit) {
    decision = Decision.PAUSE; // (R1)
    message = messages.get("intervention.hold");
} else if (watchlistHit) {
    decision = Decision.WARN; // (R2)
    message = messages.get("intervention.watch");
} else {
    decision = Decision.ALLOW; // (R3)
    message = messages.get("intervention.clear");
}
// Ghi lại su kien can thiep (side effect!)
InterventionEvent event = new InterventionEvent(
    UUID.randomUUID(), tenantId, request.userId(),
    String.join(",", signals), decision, Instant.now());
eventRepository.save(event);
return new InterventionDecision(
    decision.name(), signals, message);
}
```

Chiến lược test: mỗi nhánh một test method

Rule	onCall	newPayee	watchlistHit	Decision
R1	T	T	T	PAUSE
R2	F	F	T	WARN
R3a	T	T	F	ALLOW (bẫy hay!)
R3b	F	F	F	ALLOW, signals rỗng

R2 dùng F/F làm đại diện cho "không đồng thời cả hai = T".

- Điều khiển `onCall`, `newPayee`: qua **tham số** `EvaluateRequest` — không cần mock.
- Điều khiển `watchlistHit`: qua **stub** `watchRepository` trả `Optional.of(...)` hay `empty()`.
- Mỗi test còn phải verify **side effect**: một `InterventionEvent` đúng nội dung được lưu.

Liên hệ Buổi 3–4: đây chính là phủ nhánh (branch coverage) của CFG `evaluate()` — nay hiện thực bằng Mockito thay vì chỉ vẽ trên giấy.

Test R1 — PAUSE (1/2): Arrange + Act

Trích nguyên văn test thật:

```
@Test
void evaluate_whenOnCallAndNewPayeeAndWatchlistHit_pausesWithEducationalMessage()
{
    // Arrange
    UUID userId = UUID.randomUUID();
    AccountWatchEntry watchHit = new AccountWatchEntry(
        UUID.randomUUID(), TENANT_ID,
        WatchType.BANK_ACCOUNT, DEST_ACCOUNT,
        RiskLevel.CONFIRMED, "fraud-feed");
    when(watchRepository
        .findByTenantIdAndValue(TENANT_ID, DEST_ACCOUNT))
        .thenReturn(Optional.of(watchHit));
    EvaluateRequest request = new EvaluateRequest(
        userId, new BigDecimal("5000000"),
        DEST_ACCOUNT, true, true);    // onCall, newPayee

    // Act
    InterventionDecision decision =
        interceptionService.evaluate(request);
}
```

Test R1 — PAUSE (2/2): Assert hai tầng

```
// Assert 1: gia tri tra ve (state verification)
assertThat(decision.decision())
    .isEqualTo(Decision.PAUSE.name());
assertThat(decision.signals()).containsExactly(
    "ON_CALL", "NEW_PAYEE", "WATCHLIST_HIT");
assertThat(decision.message()).isNotBlank();

// Assert 2: su kien duoc luu (behaviour verification)
verify(eventRepository).save(eventCaptor.capture());
InterventionEvent persisted = eventCaptor.getValue();
assertThat(persisted.getTenantId()).isEqualTo(TENANT_ID);
assertThat(persisted.getUserId()).isEqualTo(userId);
assertThat(persisted.getDecision())
    .isEqualTo(Decision.PAUSE);
assertThat(persisted.getSignals())
    .isEqualTo("ON_CALL,NEW_PAYEE,WATCHLIST_HIT");
}
```

Một test — hai góc nhìn

Kết quả trả về đúng **và** dữ liệu ghi xuống DB đúng.

Test R2 — WARN: hit nhưng không đủ điều kiện PAUSE

```
@Test
void evaluate_whenWatchlistHitButNotOnCallNorNewPayee_warns() {
    // Arrange: van hit watchlist, nhưng onCall=false,
    // newPayee=false
    UUID userId = UUID.randomUUID();
    AccountWatchEntry watchHit = new AccountWatchEntry(
        UUID.randomUUID(), TENANT_ID,
        WatchType.BANK_ACCOUNT, DEST_ACCOUNT,
        RiskLevel.HIGH, "fraud-feed");
    when(watchRepository
        .findByTenantIdAndValue(TENANT_ID, DEST_ACCOUNT))
        .thenReturn(Optional.of(watchHit));
    EvaluateRequest request = new EvaluateRequest(
        userId, new BigDecimal("100"),
        DEST_ACCOUNT, false, false);
    InterventionDecision decision =
        interceptionService.evaluate(request);
    assertThat(decision.decision())
        .isEqualTo(Decision.WARN.name());
    assertThat(decision.signals())
        .containsExactly("WATCHLIST_HIT");
}
```

Test R3a — ALLOW: tín hiệu hành vi, tài khoản sạch

```
@Test
void evaluate_whenOnCallAndNewPayeeButNoWatchlistHit_warnsIsNotTriggeredAndAllows
() {
    // Arrange: tin hieu cao nhung KHONG nam trong watchlist
    UUID userId = UUID.randomUUID();
    when(watchRepository
        .findByTenantIdAndValue(TENANT_ID, DEST_ACCOUNT))
        .thenReturn(Optional.empty());
    EvaluateRequest request = new EvaluateRequest(
        userId, new BigDecimal("9999"),
        DEST_ACCOUNT, true, true);
    InterventionDecision decision =
        interceptionService.evaluate(request);
    // Không hit -> rơi xuống nhanh ALLOW
    assertThat(decision.decision())
        .isEqualTo(Decision.ALLOW.name());
    assertThat(decision.signals())
        .containsExactly("ON_CALL", "NEW_PAYEE");
}
```

Vì sao test này quý? Nó chứng minh rule **không** PAUSE/WARN chỉ vì hành vi đáng ngờ — tránh false positive làm phiền người dùng.

Test R3b — ALLOW: không tín hiệu, signals rỗng

```
@Test
void evaluate_whenNoSignals_allowsAndPersistsEmptySignals() {
    UUID userId = UUID.randomUUID();
    when(watchRepository
        .findByTenantIdAndValue(TENANT_ID, DEST_ACCOUNT))
        .thenReturn(Optional.empty());
    InterventionDecision decision = interceptionService
        .evaluate(new EvaluateRequest(userId,
            new BigDecimal("10"), DEST_ACCOUNT,
            false, false));
    assertThat(decision.signals()).isEmpty();
    verify(eventRepository).save(eventCaptor.capture());
    assertThat(eventCaptor.getValue().getSignals())
        .isEmpty();    // chuỗi signals rỗng được lưu
}
```

Test checkAccount — ủy quyền đúng tenant

```
@Test
void checkAccount_delegatesToRepositoryScopedByTenant() {
    AccountWatchEntry entry = new AccountWatchEntry(
        UUID.randomUUID(), TENANT_ID, WatchType.PHONE,
        "+849000000000", RiskLevel.WATCH, "user-report");
    when(watchRepository.findByTenantIdAndValue(
        TENANT_ID, "+849000000000"))
        .thenReturn(Optional.of(entry));
    assertThat(interceptionService
        .checkAccount("+849000000000")).containsSame(entry);
    verify(watchRepository)
        .findByTenantIdAndValue(TENANT_ID, "+849000000000");
}
```

Tổng kết ca kiểm thử evaluate() (1/2)

Test method (thật trong repo)	Nhánh	Kỹ thuật chính
...WatchlistHit_pauses...	R1 PAUSE	stub hit + captor sự kiện
...ButNotOnCallNorNewPayee	R2 WARN	stub hit, request F/F
_warns		
...ButNoWatchlistHit	R3a ALLOW	stub
_...Allows		Optional.empty()
...whenNoSignals_allows...	R3b ALLOW	signals rỗng được lưu
checkAccount_delegates...	ủy quyền	verify tham số tenant

Tổng kết ca kiểm thử evaluate() (2/2)

- 5 test → 100% **decision coverage** cho evaluate() + checkAccount().
- Lưu ý: JaCoCo đếm branch theo **từng điều kiện con** của && (short-circuit) nên report có thể báo thiếu 1 branch (newPayee=F khi onCall=T); muốn 100% branch theo JaCoCo → thêm một test (T, F, T).
- Mỗi test độc lập: tự stub, tự tạo request, không phụ thuộc thứ tự.
- Toàn bộ chạy < 1 giây: ./gradlew :modules:interception:test.

verify — đếm số lần gọi

```
// Mac dinh = times(1): duoc goi dung mot lan
verify(eventRepository).save(any(InterventionEvent.class));

verify(eventRepository, times(1)).save(any());
verify(eventRepository, never()).deleteById(any());
verify(watchRepository, atLeastOnce())
    .findByTenantIdAndValue(any(), anyString());
verify(watchRepository, atMost(2))
    .findByTenantIdAndValue(any(), anyString());
```

Chế độ	Ý nghĩa
<code>times(n)</code>	đúng n lần (mặc định $n = 1$)
<code>never()</code>	không được gọi lần nào
<code>atLeast(n) / atLeastOnce()</code>	tối thiểu $n / 1$ lần
<code>atMost(n)</code>	tối đa n lần

verifyNoInteractions và InOrder

```
// Mock KHÔNG được dùng đến (gọi bất kỳ method nào -> fail)
verifyNoInteractions(eventPublisher);

// Sau các verify đã liệt kê, không còn lỗi gọi nào khác
verifyNoMoreInteractions(eventRepository);

// Kiểm tra THU TU: lưu báo cáo trước, publish event sau
InOrder inOrder = inOrder(reportRepository, eventPublisher);
inOrder.verify(reportRepository)
    .save(any(PhishingReport.class));
inOrder.verify(eventPublisher)
    .publish(any(PhishingReportConfirmedEvent.class));
```

Khi nào cần InOrder?

Khi thứ tự là một phần của hợp đồng nghiệp vụ — ví dụ phải **lưu xong** báo cáo rồi mới phát sự kiện (listener sẽ đọc lại từ DB). Nếu thứ tự không quan trọng, đừng ràng buộc: test sẽ dễ vỡ.

Nguyên tắc: failure path phải verify điều KHÔNG xảy ra (1/2)

Ví dụ `ReportingServiceImpl.triage(id, false)` — bác bỏ báo cáo:

```
@Test
void triage_whenDismissed_doesNotPublishEvent() {
    when(reportRepository.findById(reportId))
        .thenReturn(Optional.of(report));
    when(reportRepository.save(any(PhishingReport.class)))
        .thenAnswer(inv -> inv.getArgument(0));

    reportingService.triage(reportId, false); // dismiss

    // Diem mau chot: KHONG duoc phat event "da xac nhan"
    verifyNoInteractions(eventPublisher);
}
```

Nguyên tắc: failure path phải verify điều KHÔNG xảy ra (2/2)

Nếu thiếu dòng verify này?

Bug "dismiss vẫn publish PhishingReportConfirmedEvent" sẽ lọt lưới: analytics tưởng báo cáo đã được xác nhận nên **giảm** điểm rủi ro của người dùng 15 điểm (weight -15) — giảm sai, vì báo cáo thực ra bị bác bỏ → hệ thống đánh giá người dùng "cảnh giác" hơn thực tế. Trong khi đó mọi assert trên giá trị trả về vẫn xanh.

Vấn đề: đối tượng sinh ra bên trong service (1/2)

```
// Trong evaluate(): event được new ngay trong service
InterventionEvent event = new InterventionEvent(
    UUID.randomUUID(), tenantId, request.userId(),
    String.join(",", signals), decision, Instant.now());
eventRepository.save(event);
```

- Test **không có tham chiếu** tới event — nó không được trả về, không truyền vào từ ngoài.
- `verify(eventRepository).save(any())` chỉ biết "có lưu gì đó", không biết **nội dung** đúng hay sai.
- Cần "bắt" argument tại thời điểm mock được gọi → `ArgumentCaptor`.

Vấn đề: đối tượng sinh ra bên trong service (2/2)

Hai cách tạo captor

```
@Captor ArgumentCaptor<InterventionEvent> eventCaptor;
```

(field, như test thật)
hoặc `ArgumentCaptor.forClass(InterventionEvent.class)`
(cục bộ trong test).

Captor trong test thật — bắt InterventionEvent

```
@Captor
private ArgumentCaptor<InterventionEvent> eventCaptor;

// ... trong test method:
verify(eventRepository).save(eventCaptor.capture());
InterventionEvent persisted = eventCaptor.getValue();

assertThat(persisted.getId()).isNotNull();
assertThat(persisted.getTenantId()).isEqualTo(TENANT_ID);
assertThat(persisted.getUserId()).isEqualTo(userId);
assertThat(persisted.getDecision())
    .isEqualTo(Decision.PAUSE);
assertThat(persisted.getSignals())
    .isEqualTo("ON_CALL,NEW_PAYEE,WATCHLIST_HIT");
assertThat(persisted.getTs()).isNotNull();
```

Đây chính là Data Flow Testing (Buổi 4) bằng công cụ

Kiểm tra dữ liệu từ điểm định nghĩa (request, signals) tới điểm sử dụng (bản ghi được lưu) — không chỉ kiểm tra nhánh.

getValue() vs getAllValues()

```
// importUsers() gọi createUser() -> save() NHIEU lần  
authService.importUsers(List.of(u1, u2, u3));  
  
ArgumentCaptor<AppUser> captor =  
    ArgumentCaptor.forClass(AppUser.class);  
verify(userRepository, times(3)).save(captor.capture());  
  
List<AppUser> saved = captor.getAllValues();  
assertThat(saved).hasSize(3);  
assertThat(saved).extracting(AppUser::getEmail)  
    .containsExactly("a@dgs.vn", "b@dgs.vn", "c@dgs.vn");
```

getValue()	argument của lần gọi cuối cùng
getAllValues()	danh sách argument của mọi lần gọi

Khi nào dùng captor, khi nào assert giá trị trả về?

Assert giá trị trả về khi...

- Method **trả về** kết quả cần kiểm tra (`InterventionDecision`)
- Dữ liệu quan sát được từ "bên ngoài" service

Dùng ArgumentCaptor khi...

- Đối tượng chỉ đi **vào dependency** (`save`, `publish`, `send`)
- Cần kiểm tra **nội dung** chi tiết, không chỉ "có gọi"
- Giá trị sinh nội bộ: `id`, `timestamp`, chuỗi `signals` ghép

Thứ tự ưu tiên

(1) assert giá trị trả về → (2) captor cho side effect → (3) verify đếm lần gọi. Đừng dùng captor khi một `assertThat(result)` đơn giản là đủ — test ngắn nhất có thể.

Ca kiểm thử: AuthServiceImpl.createUser()

```
@Override
@Transactional
public UserView createUser(UserUpsert input) {
    UUID tenantId = TenantContext.requireUuid();
    String email = input.email();
    if (email == null || email.isBlank()) {
        throw new IllegalArgumentException(
            "email is required");
    }
    AppUser existing = userRepository
        .findByTenantIdAndEmail(tenantId, email)
        .orElse(null);
    if (existing != null) { // idempotent: cập nhật
        applyChanges(existing, input);
        return toUserView(userRepository.save(existing));
    }
    // ... tạo user mới, role mặc định LEARNER ...
}
```

DigiShield không dùng bean validation: không có @NotNull/@Email — mọi ràng buộc nằm trong code service → phải test bằng unit test, không dựa vào framework.

Ba mức kiểm thử exception

Mức 1: đúng loại exception (JUnit assertThrows)

```
assertThrows(IllegalArgumentException.class,  
    () -> authService.createUser(  
        new UserUpsert(" ", null, null, null)));
```

Mức 2: đúng loại + đúng message

```
IllegalArgumentException ex = assertThrows(  
    IllegalArgumentException.class,  
    () -> authService.createUser(blankEmailInput));  
assertEquals("email is required", ex.getMessage());
```

Mức 3: + không có side effect

```
verify(userRepository, never()).save(any(AppUser.class));
```

Phong cách AssertJ: assertThatThrownBy

```
@Test
void createUser_whenEmailBlank_throwsAndSavesNothing() {
    // Arrange: service doc tenant tu TenantContext
    TenantContext.set(TENANT_ID.toString());
    UserUpsert input = new UserUpsert(" ", "learner",
                                     null, "vi");

    // Act + Assert: chuỗi fluent của AssertJ
    assertThatThrownBy(() -> authService.createUser(input))
        .assertInstanceOf(IllegalArgumentException.class)
        .hasMessage("email is required");

    // Không được dùng chạm gì đến repository sau validate
    verify(userRepository, never()).save(any());
}
```

So với assertThrows

Cùng sức mạnh; AssertJ cho phép nối `.hasMessageContaining()`, `.hasCauseInstanceOf()`... trong một chuỗi, đồng bộ với style `assertThat` của phần còn lại trong repo.

Ca kiểm thử:

AnalyticsServiceImpl.recomputeRisk() (1/2)

Service tính điểm rủi ro rồi **phát sự kiện** cho module khác:

```
private RiskScore doRecompute(UUID tenantId, UUID userId) {  
    int value = computeScore(tenantId, userId);  
  
    RiskScore score = new RiskScore(  
        UUID.randomUUID(), tenantId, RiskScope.USER,  
        userId, value, Instant.now());  
    RiskScore saved = riskScoreRepository.save(score);  
  
    eventPublisher.publish(  
        new RiskRecomputedEvent(tenantId, userId, value));  
    return saved;  
}
```

Ca kiểm thử:

`AnalyticsServiceImpl.recomputeRisk()` (2/2)

`EventPublisher` là một interface

DigiShield bọc `ApplicationEventPublisher` của Spring sau interface `shared.messaging.EventPublisher` → mock được như mọi dependency khác, không cần Spring context.

Test thật (trích AnalyticsServiceImplTest) — 1/2

```
@Mock private RiskScoreRepository riskScoreRepository;
@Mock private RiskSignalRepository riskSignalRepository;
@Mock private EventPublisher eventPublisher;
@InjectMocks private AnalyticsServiceImpl analyticsService;

@Captor private ArgumentCaptor<RiskScore> scoreCaptor;
@Captor private ArgumentCaptor<RiskRecomputedEvent>
    eventCaptor;

@Test
void recomputeRisk_persistsUserScopedScoreAndPublishesEvent() {
    UUID userId = UUID.randomUUID();
    when(riskSignalRepository
        .findByTenantIdAndUserIdAndOccurredAtAfter(
            eq(TENANT_ID), eq(userId), any(Instant.class)))
        .thenReturn(List.of()); // không có tin hiệu
    when(riskScoreRepository.save(any(RiskScore.class)))
        .thenAnswer(inv -> inv.getArgument(0));

    RiskScore result = analyticsService
        .recomputeRisk(userId);
```

Test thật: điểm được lưu và sự kiện được phát (2/2)

```
// Assert: entity được lưu đúng
verify(riskScoreRepository).save(scoreCaptor.capture());
RiskScore persisted = scoreCaptor.getValue();
assertThat(persisted.getScope())
    .isEqualTo(RiskScope.USER);
assertThat(persisted.getScopeId()).isEqualTo(userId);
assertThat(persisted.getValue()).isEqualTo(5);
// không tin hiệu -> điểm nên = BASE_RISK
assertThat(result).isSameAs(persisted);
// Assert: sự kiện phản chiếu đúng điểm đã lưu
verify(eventPublisher).publish(eventCaptor.capture());
RiskRecomputedEvent event = eventCaptor.getValue();
assertThat(event.tenantId()).isEqualTo(TENANT_ID);
assertThat(event.userId()).isEqualTo(userId);
assertThat(event.score())
    .isEqualTo(persisted.getValue());
}
```

Mẫu tái dùng cho BTL: cặp captor “entity lưu xuống” + “event phát ra”, rồi assert **chéo**: event phải khớp dữ liệu đã lưu.

@Nested — nhóm test theo kịch bản

```
@ExtendWith(MockitoExtension.class)
class InterceptionServiceImplTest {
    // ... @Mock / @InjectMocks như trên ...
    @Nested
    @DisplayName("Khi tài khoản dịch nằm trong watchlist")
    class WhenWatchlistHit {
        @BeforeEach                // chỉ chạy cho nhóm này
        void stubHit() {
            when(repository.findByTenantIdAndValue(
                TENANT_ID, DEST_ACCOUNT))
                .thenReturn(Optional.of(watchHit));
        }
        @Test @DisplayName("onCall + newPayee -> PAUSE")
        void pausesWhenBothBehaviourSignals() { ... }
        @Test @DisplayName("không có tin hiệu -> WARN")
        void warnsOtherwise() { ... }
    }
    @Nested
    @DisplayName("Khi tài khoản dịch sạch")
    class WhenAccountClean { ... } // 2 test ALLOW
}
```

Lợi ích của @Nested + @DisplayName

Báo cáo test trong IDE / Gradle:

InterceptionServiceImplTest

Khi tài khoản đích nằm trong watchlist

onCall + newPayee → PAUSE

không có tín hiệu → WARN

Khi tài khoản đích sạch

luôn ALLOW, signals đúng

Ghi nhớ:

- @BeforeEach trong nhóm chỉ chạy cho nhóm đó → hết lặp stub
- @DisplayName tiếng Việt: báo cáo đọc được bởi cả người không code
- Khuyến nghị dùng khi class ≥ 10 test method

Helper builder cho dữ liệu test

EvaluateRequest có 5 tham số — lặp lại ở mọi test rất ồn:

```
// Helper trong test class: chỉ tham số QUAN TRỌNG là ra
private EvaluateRequest request(boolean onCall,
                                boolean newPayee) {
    return new EvaluateRequest(
        UUID.randomUUID(),           // userId bất kỳ
        new BigDecimal("1000000"),   // số tiền mặc định
        DEST_ACCOUNT, onCall, newPayee);
}

// Test đọc thành một câu:
interceptionService.evaluate(request(true, true));
interceptionService.evaluate(request(false, false));
```

Nguyên tắc "test data minimal"

Trong mỗi test chỉ hiện các giá trị **ảnh hưởng đến kết quả** (onCall, newPayee); phần còn lại giấu vào helper. Người đọc thấy ngay input nào quyết định output nào.

@Mock vs @MockitoBean (Spring)

@Mock (Mockito thuần)

- Không Spring context
- @ExtendWith(MockitoExtension.class)
- Tốc độ: **mili-giây**
- Dùng cho: **unit test**

```
@ExtendWith(
    MockitoExtension.class)
class InterceptionServiceImplTest {
    @Mock
    AccountWatchEntryRepository repo;
    @InjectMocks
    InterceptionServiceImpl svc;
}
```

@MockitoBean (Spring Boot 4)

- Thay bean **trong** **ApplicationContext**
- Dùng với @SpringBootTest
- Tốc độ: **vài giây** (khởi động context)
- Dùng cho: **integration test**

```
@SpringBootTest
class InterceptionControllerIT {
    @MockitoBean
    AiClient aiClient;    // thay bean
    @Autowired
    MockMvc mockMvc;    // bean that
}
```

Lưu ý phiên bản

@MockBean (cũ) đã bị thay bởi @MockitoBean từ Spring Boot 3.4+; DigiShield chạy Spring Boot 4 nên chỉ dùng @MockitoBean.

Khi nào mới cần Spring context? (dẫn sang Buổi 7)

Tình huống	@Mock	@MockitoBean
Unit test logic một service	✓	
Test controller qua MockMvc		✓
Test security/filter/serialization		✓
Test listener sự kiện Spring Modulith		✓
Test truy vấn JPA thật (Testcontainers)		✓
Pipeline CI cần nhanh, chạy mỗi commit	✓	

Trong DigiShield: `./gradlew test` — unit test Mockito, không cần Docker, vài giây; `./gradlew integrationTest` — `*IT.java` với `@SpringBootTest` + `Testcontainers`, cần Docker (Buổi 7).

Testing pyramid (Buổi 2): nhiều unit test rẻ ở đáy, ít integration test đắt ở giữa — tỷ lệ trong repo hiện tại: 176 unit / 19 integration.

Anti-pattern 1–2: over-mocking, test chép lại cài đặt

Over-mocking

Mock cả những thứ không cần: DTO, entity, List, class tiện ích. EvaluateRequest là record — cứ new nó ra! Chỉ mock **dependency có side effect hoặc khó dựng** (DB, mạng, thời gian).

Test chép lại cài đặt (mirror implementation)

```
// XAU: lap lai cong thuc cua service trong test
int expected = Math.max(0, Math.min(100, 5 + weight));
assertThat(result.getValue()).isEqualTo(expected);

// TOT (nhu test that): tinh tay ky vong tu dac ta
// 2 click x 25 diem -> 5 + 50 = 55
assertThat(result.getValue()).isEqualTo(55);
```

Copy công thức vào test → code sai thì test cũng sai theo, không bao giờ fail. Kỳ vọng phải đến từ **đặc tả**, không từ code.

Anti-pattern 3–4: mock "trả lời mọi thứ", assert quá ít

Mock trả lời mọi thứ

```
// XAU: any() moi noi -> test khong con kiem tra
// service truyen DUNG tenant/tai khoan hay khong
when(watchRepository.findByTenantIdAndValue(
    any(), any())).thenReturn(Optional.of(hit));
```

Test thật của DigiShield stub với TENANT_ID, DEST_ACCOUNT cụ thể: nếu service truyền nhầm tenant, stub không khớp → test fail. Strict stubbing chỉ cứu được một phần — hãy stub hẹp nhất có thể.

Assert quá ít (test tăng coverage rỗng)

```
// XAU: gọi xong không assert gì cả
interceptionService.evaluate(request);    // "no crash" ??
```

Coverage 100% nhưng không phát hiện lỗi nào. Mutation testing (PIT) sẽ vạch trần loại test này — mỗi test tối thiểu một assert có ý nghĩa.

Tổng quan bài thực hành Buổi 6

BT	Nội dung	Kỹ thuật	Phút
1	Chạy và đọc bộ test có sẵn của interception	<code>./gradlew test</code> , đọc report	10
2	<code>TestReportingServiceImpl.triage()</code> — nhánh confirm	stub, captor, verify publish	20
3	<code>triage()</code> — nhánh dismiss + cross-tenant	never, <code>verifyNoInteractions</code> , <code>assertThatThrownBy</code>	20
4	<code>TestAuthServiceImpl.importUsers()</code>	<code>getAllValues()</code> , đếm accepted	20
5	Đo coverage các class vừa test	JaCoCo report	10
Tổng			80

Chuẩn bị: mở `modules/reporting/.../ReportingServiceImpl.java` và `modules/auth/.../AuthServiceImpl.java`; test đặt trong `src/test/java` cùng package với class được test.

BT 2–3: mã nguồn triage() cần test

```
public PhishingReport triage(UUID reportId,
                             boolean confirmThreat) {
    UUID tenantId = TenantContext.requireUuid();
    PhishingReport report = reportRepository
        .findById(reportId)
        .filter(r -> tenantId.equals(r.getTenantId()))
        .orElseThrow(() -> new IllegalArgumentException(
            "Không tìm thấy báo cáo phishing: " + reportId));

    if (confirmThreat) {
        report.setAiLabel(AiLabel.THREAT);
        report.setStatus(ReportStatus.CONFIRMED);
        PhishingReport saved = reportRepository.save(report);
        eventPublisher.publish(
            new PhishingReportConfirmedEvent(
                tenantId, saved.getUserId(), saved.getId()));
        return saved;
    }
    report.setAiLabel(AiLabel.CLEAN);
    report.setStatus(ReportStatus.DISMISSED);
    return reportRepository.save(report);
}
```

BT 2–3: khung test cần hoàn thiện (1/2)

```
@ExtendWith(MockitoExtension.class)
class ReportingServiceImplTest {
    @Mock PhishingReportRepository reportRepository;
    @Mock BlacklistEntryRepository blacklistRepository;
    @Mock ThreatIntelRepository threatIntelRepository;
    @Mock EventPublisher eventPublisher;
    @InjectMocks ReportingServiceImpl reportingService;
    @Captor ArgumentCaptor<PhishingReportConfirmedEvent>
        eventCaptor;

    @Test
    void triage_confirm_marksThreatAndPublishesEvent() {
        // TODO: stub findById tra ve report cung tenant
        // TODO: stub save tra ve argument (thenAnswer)
        // TODO: goi triage(id, true)
        // TODO: assert status CONFIRMED, aiLabel THREAT
        // TODO: captor event -- dung tenantId/userId/reportId
    }
}
```

BT 2–3: khung test cần hoàn thiện (2/2)

```
@Test
void triage_dismiss_marksCleanAndDoesNotPublish() {
    // TODO: gọi triage(id, false)
    // TODO: assert DISMISSED / CLEAN
    // TODO: verifyNoInteractions(eventPublisher)
}

@Test
void triage_reportOfOtherTenant_throwsAndSavesNothing() {
    // TODO: report.tenantId KHÁC TenantContext
    // TODO: assertThatThrownBy ... IllegalArgumentException
    // TODO: verify(reportRepository, never()).save(any())
}
}
```

Lưu ý: repo đã có `ReportingServiceImplTest` hoàn chỉnh (5 test) và `AuthServiceImplTest` (19 test) — tự viết theo khung trên *trước*, rồi mới mở file thật để so đáp án (tên test trong repo dài và mô tả kỹ hơn).

BT 4: importUsers() — đếm accepted, bỏ dòng blank

```
public ImportResult importUsers(List<UserUpsert> users) {
    int accepted = 0;
    if (users != null) {
        for (UserUpsert input : users) {
            if (input == null || input.email() == null
                || input.email().isBlank()) {
                continue;           // bỏ qua dòng hong
            }
            createUser(input);
            accepted++;
        }
    }
    String jobId = "import-" + UUID.randomUUID();
    return new ImportResult(jobId, accepted);
}
```

Yêu cầu tối thiểu 4 test: (1) 3 dòng hợp lệ → accepted = 3, save 3 lần, getAllValues() đúng email; (2) lần dòng null/email blank → chỉ đếm dòng hợp lệ; (3) danh sách null → accepted = 0, không save; (4) jobId có tiền tố "import-".

Chạy test và đo coverage

```
# Chạy toàn bộ unit test (không cần Docker)
./gradlew test

# Chạy riêng một module / một class
./gradlew :modules:reporting:test
./gradlew :modules:auth:test \
    --tests "*AuthServiceImplTest"

# Báo cáo coverage tổng hợp (JaCoCo)
./gradlew testCodeCoverageReport
# Mo: build/reports/jacoco/
#      testCodeCoverageReport/html/index.html
```

Kiểm tra chéo: sau khi viết đủ test, `trriage()` và `importUsers()` phải đạt **100% branch** trong JaCoCo — JaCoCo đếm branch theo **từng điều kiện con** của `||`: cần đủ ba loại dòng hỏng (`null`, `email null`, `email blank`).

Bài nộp — gắn với Bài tập lớn (A1.2, 30%)

Yêu cầu (theo nhóm BTL)

- 1 Bộ **unit test Mockito** cho service chính của module nhóm đang phụ trách (interception, reporting, auth, analytics, simulation...):
 - Mỗi nhánh quyết định một test method (như `evaluate()`)
 - Có ít nhất: 1 ArgumentCaptor, 1 test exception (type + message + side effect), 1 verify publish sự kiện hoặc `verifyNoInteractions`
 - Tổ chức bằng `@Nested` + `@DisplayName` tiếng Việt
- 2 Ảnh chụp **JaCoCo report**: branch coverage của service được test $\geq 80\%$; ghi rõ nhánh nào chưa phủ và vì sao.
- 3 Bảng ánh xạ: test method $\leftrightarrow$ nhánh/rule $\leftrightarrow$ kỹ thuật (stub / captor / verify) — như slide "Tổng kết ca kiểm thử".

Định dạng: push lên nhánh `feat/btl-unit-test-<nhóm>` của repo nhóm + nộp PDF báo cáo `Nhom_XX_Buoi06.pdf`. Hạn: trước Buổi 7.

Câu hỏi ôn tập

- 1 Phân biệt stub và mock theo Meszaros. Trong `InterceptionServiceImplTest`, `watchRepository` đóng vai trò nào ở test PAUSE?
- 2 Vì sao `@BeforeEach` của test đó phải dùng `lenient()`? Bỏ đi thì test nào sẽ fail và với exception gì?
- 3 Viết lại `when(repo.findByTenantIdAndValue(TENANT_ID, any()))` cho đúng quy tắc matcher.
- 4 Khi nào bắt buộc dùng `ArgumentCaptor` thay vì `assert` giá trị trả về? Cho ví dụ từ `evaluate()`.
- 5 Test "dismiss không publish event" bảo vệ chúng ta khỏi bug nghiệp vụ nào trong chuỗi reporting → analytics?
- 6 `@Mock` và `@MockitoBean` khác nhau thế nào? Nếu dùng `@MockitoBean` trong test không có `@SpringBootTest` thì sao?

Tóm tắt Buổi 6 và chuẩn bị Buổi 7

Đã học

- Test double: dummy/stub/fake/spy/mock
- @Mock, @InjectMocks, @Captor, lenient()
- Stub: thenReturn/thenThrow/thenAnswer, matcher
- Ca kiểm thử evaluate(): 5 test phủ 3 nhánh
- verify, verifyNoInteractions, InOrder
- Captor cho entity và event; exception 3 mức
- @Nested, anti-patterns

Buổi 7: Integration test

- @SpringBootTest, MockMvc
- Testcontainers (PostgreSQL)
- Test Row-Level Security đa tenant
- Test pipeline sự kiện AIDA

Chuẩn bị

Cài **Docker Desktop**; chạy thử integrationTest trước ở nhà (lần đầu kéo image PostgreSQL 16 khá lâu).

Tài liệu tham khảo

- [1] Slide bài giảng điện tử.
- [2] Paul Ammann & Jeff Offutt, *Introduction to Software Testing*, Cambridge University Press, 2008.
- [3] Glenford J. Myers, *The Art of Software Testing*, 2nd ed., Wiley, 2004.
- [4] ISTQB, *Certified Tester Foundation Level Syllabus*, version 4.0, 2023.
<https://istqb.org/>
- [5] Mockito Team, *Mockito Documentation*. <https://site.mockito.org/>
- [6] JUnit Team, *JUnit 5 User Guide*.
<https://junit.org/junit5/docs/current/user-guide/>
- [7] Gerard Meszaros, *xUnit Test Patterns: Refactoring Test Code*, Addison-Wesley, 2007.
- [8] Mã nguồn DigiShield: `modules/interception`, `modules/auth`,
`modules/analytics`, `modules/reporting` (test trong `src/test/java`).